Turtlesim "Catch Them All" Project (ROS 2)

Built using:

- ROS 2 Humble
- Python
- Turtlesim

This project creates an autonomous turtle that:

- Spawns enemy turtles randomly
- Detects nearest turtle
- Chases it
- Kills it automatically

Project Structure

```
~/turtlesim_ws/
├── src/
│   └── turtle_catcher/
│       ├── launch/
│       │   └── turtle_game.launch.py
│       ├── resource/
│       ├── turtle_catcher/
│       │   ├── __init__.py
│       │   ├── turtle_controller.py
│       │   ├── turtle_pose_subscriber.py
│       │   └── turtle_spawner.py
│       ├── package.xml
│       └── setup.py
```

STEP 1 — Create Workspace

```
mkdir -p ~/turtlesim_ws/src
cd ~/turtlesim_ws/src
```

STEP 2 — Create Package

```
ros2 pkg create turtle_catcher --build-type ament_python --dependencies rclpy
turtlesim geometry_msgs std_msgs
```

STEP 3 — Create turtle_controller.py

Go to package folder:

```
cd ~/turtlesim_ws/src/turtle_catcher/turtle_catcher
```

Create file:

```
touch turtle_controller.py
chmod +x turtle_controller.py
```

Paste this code:

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
from turtlesim.srv import Kill

import math


class TurtleController(Node):

    def __init__(self):

        super().__init__("turtle_controller")

        self.cmd_vel_pub = self.create_publisher(
            Twist,
            "/turtle1/cmd_vel",
            10
```

```
        )

        self.pose_subscriber = self.create_subscription(
            Pose,
            "/turtle1/pose",
            self.pose_callback,
            10
        )

        self.kill_client = self.create_client(
            Kill,
            "/kill"
        )

        while not self.kill_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info("Waiting for kill service...")

        self.current_pose = None

        self.alive_turtles = {}

        for i in range(2, 20):

            turtle_name = f"turtle{i}"

            self.create_subscription(
                Pose,
                f"/{turtle_name}/pose",
                lambda msg, name=turtle_name:
                self.target_pose_callback(msg, name),
                10
            )

        self.timer = self.create_timer(
            0.1,
            self.control_loop
        )

        self.get_logger().info("Turtle controller started")

    def pose_callback(self, msg):

        self.current_pose = msg

    def target_pose_callback(self, msg, turtle_name):

        self.alive_turtles[turtle_name] = msg
```

```
    def control_loop(self):

        if self.current_pose is None:
            return

        if len(self.alive_turtles) == 0:
            return

        nearest_turtle = None
        nearest_distance = 999999.0

        for turtle_name, pose in self.alive_turtles.items():

            distance = math.sqrt(
                (pose.x - self.current_pose.x) ** 2 +
                (pose.y - self.current_pose.y) ** 2
            )

            if distance < nearest_distance:

                nearest_distance = distance
                nearest_turtle = turtle_name

        if nearest_turtle is None:
            return

        target_pose = self.alive_turtles[nearest_turtle]

        angle_to_target = math.atan2(
            target_pose.y - self.current_pose.y,
            target_pose.x - self.current_pose.x
        )

        angle_error = angle_to_target - self.current_pose.theta

        while angle_error > math.pi:
            angle_error -= 2 * math.pi

        while angle_error < -math.pi:
            angle_error += 2 * math.pi

        msg = Twist()

        msg.linear.x = 2.0 * nearest_distance
        msg.angular.z = 6.0 * angle_error

        self.cmd_vel_pub.publish(msg)
```

```
        if nearest_distance < 0.5:
            self.catch_turtle(nearest_turtle)

    def catch_turtle(self, turtle_name):

        self.get_logger().info(
            f"Caught {turtle_name}"
        )

        request = Kill.Request()

        request.name = turtle_name

        future = self.kill_client.call_async(request)

        future.add_done_callback(
            lambda future:
            self.kill_done_callback(future, turtle_name)
        )

    def kill_done_callback(self, future, turtle_name):

        try:

            future.result()

            if turtle_name in self.alive_turtles:
                del self.alive_turtles[turtle_name]

            self.get_logger().info(
                f"{turtle_name} removed"
            )

        except Exception as e:

            self.get_logger().error(
                f"Kill service failed: {e}"
            )


def main(args=None):

    rclpy.init(args=args)

    node = TurtleController()

    rclpy.spin(node)
```

```
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

STEP 4 — Create turtle_pose_subscriber.py

```
cd ~/turtlesim_ws/src/turtle_catcher/turtle_catcher

touch turtle_pose_subscriber.py
chmod +x turtle_pose_subscriber.py
```

Paste this code:

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

from turtlesim.msg import Pose


class TurtlePoseSubscriber(Node):

    def __init__(self):

        super().__init__("turtle_pose_subscriber")

        self.pose_subscriber = self.create_subscription(
            Pose,
            "/turtle1/pose",
            self.pose_callback,
            10
        )

    def pose_callback(self, msg):

        self.get_logger().info(
            f"X: {msg.x:.2f}, "
            f"Y: {msg.y:.2f}, "
            f"Theta: {msg.theta:.2f}"
        )
```

```
def main(args=None):

    rclpy.init(args=args)

    node = TurtlePoseSubscriber()

    rclpy.spin(node)

    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

STEP 5 — Create turtle_spawner.py

```
cd ~/turtlesim_ws/src/turtle_catcher/turtle_catcher

touch turtle_spawner.py
chmod +x turtle_spawner.py
```

Paste this code:

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

from turtlesim.srv import Spawn

import random


class TurtleSpawner(Node):

    def __init__(self):

        super().__init__("turtle_spawner")

        self.spawn_client = self.create_client(
            Spawn,
```

```
            "/spawn"
        )

        while not self.spawn_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info("Waiting for spawn service...")

        self.turtle_counter = 2

        self.timer = self.create_timer(
            3.0,
            self.spawn_turtle
        )

        self.get_logger().info("Turtle spawner started")

    def spawn_turtle(self):

        request = Spawn.Request()

        request.x = random.uniform(1.0, 10.0)
        request.y = random.uniform(1.0, 10.0)
        request.theta = random.uniform(0.0, 6.28)

        request.name = f"turtle{self.turtle_counter}"

        self.turtle_counter += 1

        future = self.spawn_client.call_async(request)

        future.add_done_callback(self.spawn_response_callback)

    def spawn_response_callback(self, future):

        try:
            response = future.result()

            self.get_logger().info(
                f"Spawned {response.name}"
            )

        except Exception as e:
            self.get_logger().error(
                f"Service call failed: {e}"
            )


def main(args=None):
```

```
    rclpy.init(args=args)

    node = TurtleSpawner()

    rclpy.spin(node)

    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

STEP 6 — Modify setup.py

Replace the setup.py file with:

```
from setuptools import setup

package_name = 'turtle_catcher'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name]
        ),

        (
            'share/' + package_name,
            ['package.xml']
        ),

        (
            'share/' + package_name + '/launch',
            ['launch/turtle_game.launch.py']
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='user',
    maintainer_email='user@todo.todo',
```

```
    description='Turtlesim Catch Them All Project',
    license='TODO',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'turtle_controller = turtle_catcher.turtle_controller:main',
            'turtle_pose_subscriber =
turtle_catcher.turtle_pose_subscriber:main',
            'turtle_spawner = turtle_catcher.turtle_spawner:main',
        ],
    },
)
```

STEP 7 — Create Launch File

```
cd ~/turtlesim_ws/src/turtle_catcher

mkdir launch

touch launch/turtle_game.launch.py
```

Paste this code:

```
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():

    return LaunchDescription([

        Node(
            package="turtlesim",
            executable="turtlesim_node",
            name="turtlesim"
        ),

        Node(
            package="turtle_catcher",
            executable="turtle_spawner",
            name="turtle_spawner"
        ),
```

```
        Node(
            package="turtle_catcher",
            executable="turtle_controller",
            name="turtle_controller"
        ),

    ])
```

STEP 8 — Build Project

```
cd ~/turtlesim_ws
colcon build
```

STEP 9 — Source Workspace

```
source /opt/ros/humble/setup.bash
source ~/turtlesim_ws/install/setup.bash
```

STEP 10 — Run Project

```
ros2 launch turtle_catcher turtle_game.launch.py
```

GitHub Commands

Initialize git:

```
cd ~/turtlesim_ws/src/turtle_catcher

git init
```

Add files:

```
git add .
```

Commit:

```
git commit -m "Initial commit - turtlesim catch them all project"
```

Connect GitHub repo:

```
git remote add origin YOUR_GITHUB_REPO_LINK
```

Push code:

```
git branch -M main
git push -u origin main
```

Recommended .gitignore

Create file:

```
touch .gitignore
```

Paste:

```
build/
install/
log/
__pycache__/
*.pyc
```

Final Run Workflow

Every time you reopen terminal:

```
cd ~/turtlesim_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch turtle_catcher turtle_game.launch.py
```

Features Implemented

- ROS 2 Publishers
- ROS 2 Subscribers
- ROS 2 Services
- Dynamic Subscribers
- Turtle Navigation
- Proportional Controller
- Launch Files
- Multi-node Architecture
- Autonomous Catching Logic

```
~/turtlesim_ws/
├── src/
│   └── turtle_catcher/
│       ├── launch/
│       │   └── turtle_game.launch.py
│       ├── resource/
│       ├── turtle_catcher/
│       │   ├── __init__.py
│       │   ├── turtle_controller.py
│       │   ├── turtle_pose_subscriber.py
│       │   └── turtle_spawner.py
│       ├── package.xml
│       ├── setup.py
│       └── setup.cfg
```

STEP 1 — Create Workspace

```
mkdir -p ~/turtlesim_ws/src
cd ~/turtlesim_ws/src
```

STEP 2 — Create Package

```
ros2 pkg create turtle_catcher --build-type ament_python --dependencies rclpy
turtlesim geometry_msgs std_msgs
```

STEP 3 — Build Workspace

```
cd ~/turtlesim_ws
colcon build
```

STEP 4 — Source Workspace

```
source /opt/ros/humble/setup.bash
source ~/turtlesim_ws/install/setup.bash
```

STEP 5 — Create turtle_controller.py

Go to package folder:

```
cd ~/turtlesim_ws/src/turtle_catcher/turtle_catcher
```

Create file:

```
touch turtle_controller.py
chmod +x turtle_controller.py
```

---

turtle_controller.py

```
#!/usr/bin/env python3
```

```python
import rclpy
from rclpy.node import Node

from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
from turtlesim.srv import Kill

import math


class TurtleController(Node):

    def __init__(self):

        super().__init__("turtle_controller")

        # Publisher
        self.cmd_vel_pub = self.create_publisher(
            Twist,
            "/turtle1/cmd_vel",
            10
        )

        # Subscriber for turtle1 pose
        self.pose_subscriber = self.create_subscription(
            Pose,
            "/turtle1/pose",
            self.pose_callback,
            10
        )

        # Kill service client
        self.kill_client = self.create_client(
            Kill,
            "/kill"
        )

        while not self.kill_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info("Waiting for kill service...")

        # Main turtle pose
        self.current_pose = None

        # Dictionary for other turtles
        self.alive_turtles = {}

        # Create subscriptions dynamically
        for i in range(2, 20):
```

```
            turtle_name = f"turtle{i}"

            self.create_subscription(
                Pose,
                f"/{turtle_name}/pose",
                lambda msg, name=turtle_name:
                self.target_pose_callback(msg, name),
                10
            )

        # Main control loop
        self.timer = self.create_timer(
            0.1,
            self.control_loop
        )

        self.get_logger().info("Turtle controller started")

    def pose_callback(self, msg):

        self.current_pose = msg

    def target_pose_callback(self, msg, turtle_name):

        self.alive_turtles[turtle_name] = msg

    def control_loop(self):

        if self.current_pose is None:
            return

        if len(self.alive_turtles) == 0:
            return

        nearest_turtle = None
        nearest_distance = 999999.0

        # Find nearest turtle
        for turtle_name, pose in self.alive_turtles.items():

            distance = math.sqrt(
                (pose.x - self.current_pose.x) ** 2 +
                (pose.y - self.current_pose.y) ** 2
            )

            if distance < nearest_distance:
```

```
                nearest_distance = distance
                nearest_turtle = turtle_name

        if nearest_turtle is None:
            return

        target_pose = self.alive_turtles[nearest_turtle]

        # Compute angle
        angle_to_target = math.atan2(
            target_pose.y - self.current_pose.y,
            target_pose.x - self.current_pose.x
        )

        angle_error = angle_to_target - self.current_pose.theta

        # Normalize angle
        while angle_error > math.pi:
            angle_error -= 2 * math.pi

        while angle_error < -math.pi:
            angle_error += 2 * math.pi

        # Create velocity command
        msg = Twist()

        msg.linear.x = 2.0 * nearest_distance
        msg.angular.z = 6.0 * angle_error

        self.cmd_vel_pub.publish(msg)

        # Catch turtle
        if nearest_distance < 0.5:

            self.catch_turtle(nearest_turtle)

    def catch_turtle(self, turtle_name):

        self.get_logger().info(
            f"Caught {turtle_name}"
        )

        request = Kill.Request()

        request.name = turtle_name

        future = self.kill_client.call_async(request)
```

```
        future.add_done_callback(
            lambda future:
            self.kill_done_callback(future, turtle_name)
        )

    def kill_done_callback(self, future, turtle_name):

        try:

            future.result()

            if turtle_name in self.alive_turtles:
                del self.alive_turtles[turtle_name]

            self.get_logger().info(
                f"{turtle_name} removed"
            )

        except Exception as e:

            self.get_logger().error(
                f"Kill service failed: {e}"
            )


def main(args=None):

    rclpy.init(args=args)

    node = TurtleController()

    rclpy.spin(node)

    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

STEP 6 — Create turtle_pose_subscriber.py

```
touch turtle_pose_subscriber.py
chmod +x turtle_pose_subscriber.py
```

turtle_pose_subscriber.py

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

from turtlesim.msg import Pose


class TurtlePoseSubscriber(Node):

    def __init__(self):

        super().__init__("turtle_pose_subscriber")

        self.pose_subscriber = self.create_subscription(
            Pose,
            "/turtle1/pose",
            self.pose_callback,
            10
        )

    def pose_callback(self, msg):

        self.get_logger().info(
            f"X: {msg.x:.2f}, "
            f"Y: {msg.y:.2f}, "
            f"Theta: {msg.theta:.2f}"
        )


def main(args=None):

    rclpy.init(args=args)

    node = TurtlePoseSubscriber()

    rclpy.spin(node)

    rclpy.shutdown()
```

```
if __name__ == "__main__":
    main()
```

STEP 7 — Create turtle_spawner.py

```
touch turtle_spawner.py
chmod +x turtle_spawner.py
```

---

turtle_spawner.py

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

from turtlesim.srv import Spawn

import random


class TurtleSpawner(Node):

    def __init__(self):

        super().__init__("turtle_spawner")

        # Create service client
        self.spawn_client = self.create_client(
            Spawn,
            "/spawn"
        )

        # Wait until service becomes available
        while not self.spawn_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info("Waiting for spawn service...")

        # Counter for turtle names
        self.turtle_counter = 2

        # Timer to spawn turtles
```

```
        self.timer = self.create_timer(
            3.0,
            self.spawn_turtle
        )

        self.get_logger().info("Turtle spawner started")

    def spawn_turtle(self):

        # Create request object
        request = Spawn.Request()

        # Random position
        request.x = random.uniform(1.0, 10.0)
        request.y = random.uniform(1.0, 10.0)
        request.theta = random.uniform(0.0, 6.28)

        # Turtle name
        request.name = f"turtle{self.turtle_counter}"

        self.turtle_counter += 1

        # Async service call
        future = self.spawn_client.call_async(request)

        # Callback after service finishes
        future.add_done_callback(self.spawn_response_callback)

    def spawn_response_callback(self, future):

        try:
            response = future.result()

            self.get_logger().info(
                f"Spawned {response.name}"
            )

        except Exception as e:
            self.get_logger().error(
                f"Service call failed: {e}"
            )


def main(args=None):

    rclpy.init(args=args)

    node = TurtleSpawner()
```

```
    rclpy.spin(node)

    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

STEP 8 — Create Launch Folder

```
cd ~/turtlesim_ws/src/turtle_catcher
mkdir launch
```

STEP 9 — Create Launch File

```
touch launch/turtle_game.launch.py
```

---

turtle_game.launch.py

```
from launch import LaunchDescription

from launch_ros.actions import Node


def generate_launch_description():

    return LaunchDescription([

        # Turtlesim node
        Node(
            package="turtlesim",
            executable="turtlesim_node",
            name="turtlesim"
        ),

        # Turtle spawner node
```

```
        Node(
            package="turtle_catcher",
            executable="turtle_spawner",
            name="turtle_spawner"
        ),

        # Turtle controller node
        Node(
            package="turtle_catcher",
            executable="turtle_controller",
            name="turtle_controller"
        ),

    ])
```

STEP 10 — Modify setup.py

Open:

```
gedit ~/turtlesim_ws/src/turtle_catcher/setup.py
```

Replace complete file with:

```
from setuptools import setup

package_name = 'turtle_catcher'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name]
        ),

        (
            'share/' + package_name,
            ['package.xml']
        ),

        (
```

```
            'share/' + package_name + '/launch',
            ['launch/turtle_game.launch.py']
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='user',
    maintainer_email='user@todo.todo',
    description='Turtlesim Catch Them All Project',
    license='Apache License 2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'turtle_controller = turtle_catcher.turtle_controller:main',
            'turtle_pose_subscriber =
turtle_catcher.turtle_pose_subscriber:main',
            'turtle_spawner = turtle_catcher.turtle_spawner:main',
        ],
    },
)
```

STEP 11 — Build Project

```
cd ~/turtlesim_ws
colcon build
```

STEP 12 — Source Workspace

```
source /opt/ros/humble/setup.bash
source ~/turtlesim_ws/install/setup.bash
```

STEP 13 — Run Project

```
ros2 launch turtle_catcher turtle_game.launch.py
```

Optional — Add ROS Source Permanently

Open:

```
gedit ~/.bashrc
```

Add:

```
source /opt/ros/humble/setup.bash
```

Apply:

```
source ~/.bashrc
```

Future Workflow

Whenever you modify code:

```
cd ~/turtlesim_ws
colcon build
source install/setup.bash
ros2 launch turtle_catcher turtle_game.launch.py
```

GitHub Commands

Initialize git:

```
cd ~/turtlesim_ws/src/turtle_catcher
git init
```

Create README:

```
touch README.md
```

Add files:

```
git add .
```

Commit:

```
git commit -m "Initial commit for turtlesim catch them all project"
```

Connect GitHub repo:

```
git remote add origin <YOUR_GITHUB_REPO_LINK>
```

Push:

```
git branch -M main
git push -u origin main
```

Features Implemented

- ROS 2 publishers
- ROS 2 subscribers
- ROS 2 services
- Async service calls
- Dynamic subscriptions
- Robot chasing algorithm
- Turtle spawning
- Turtle killing
- Multi-node architecture
- Launch files
- ROS 2 package structure

Technologies Used

- ROS 2 Humble
- Python
- Turtlesim
- geometry_msgs
- turtlesim services

- colcon build system

Final Run Command

```
cd ~/turtlesim_ws
source install/setup.bash
ros2 launch turtle_catcher turtle_game.launch.py
```